

Large Language Models: A Multifaceted Mini-Book

Information Theory, Deep Learning, Computer Theory,
Software Engineering, Inference, and GPU Architecture

Bernard

May 24, 2026

Contents

I	Foundations	7
1	Information-Theoretic Foundations	9
1.1	Entropy and Language	9
1.2	Kullback–Leibler Divergence	10
1.3	Mutual Information in Attention	10
1.4	Rate–Distortion Theory and Quantisation	11
1.5	The Information Bottleneck and Hidden Representations	11
1.6	The Entropy of Natural Language	11
1.7	Exercises	11
2	Computational Theory of Language Models	13
2.1	The Autoregressive Paradigm	13
2.2	Computational Complexity of Attention	13
2.3	Expressiveness: What Can Transformers Compute?	14
2.4	Circuit Complexity and TC0	14
2.5	RASP: A Programming Language for Transformers	14
2.6	Injective LLMs and Input Reconstruction	15
2.7	Formal Language Hierarchy	15
2.8	The Scaling Hypothesis	15
2.9	Embedding Condensation	16
2.10	Exercises	16
II	Architecture and Training	17
3	Transformer Architecture	19
3.1	The Transformer Block	19
3.1.1	Pre-Norm vs. Post-Norm	19
3.1.2	RMSNorm	19
3.2	Masked Multi-Head Attention	20
3.3	Grouped Query Attention (GQA)	20
3.4	Feed-Forward Networks	20
3.5	RoPE: Rotary Positional Encodings	20
3.6	Attention Variants	21
3.7	Exercises	21
4	Tokenization	23
4.1	Byte-Pair Encoding (BPE)	23
4.2	SentencePiece	23

4.2.1	BPE Mode	24
4.2.2	Unigram Language Model Mode	24
4.3	tiktoken	24
4.4	WordPiece	24
4.5	Vocabulary Size Trade-offs	24
4.6	Multilingual Tokenization and CJK	25
4.7	Digit Tokenization and Arithmetic Failures	25
4.8	Special Tokens	25
4.9	Formal Semantics of BPE	26
4.10	The Tokenizer Wars (2023–2024)	26
4.11	Exercises	26
5	Mixture of Experts	27
5.1	Top- k Routing	27
5.2	Token Choice vs. Expert Choice Routing	27
5.3	Load Balancing Loss	28
5.4	Fine-Grained Experts: DeepSeekMoE	28
5.5	Expert Parallelism	28
5.6	Grouped GEMM for MoE	28
5.7	PathMoE	29
5.8	Polysemantic Experts vs. Monosemantic Paths	29
5.9	Major MoE Models	29
5.10	Exercises	30
6	Training and Scaling Laws	31
6.1	The Three-Stage Pipeline	31
6.2	Scaling Laws in Practice	31
6.3	Learning Rate Schedules	32
6.4	Distributed Training Strategies	32
6.4.1	Precision and Numerics	32
6.5	Exercises	32
7	Post-Training and Alignment	33
7.1	Supervised Fine-Tuning (SFT)	33
7.2	The RLHF Framework	33
7.3	PPO (Proximal Policy Optimization)	33
7.4	DPO (Direct Preference Optimization)	34
7.4.1	Likelihood Displacement Failure	34
7.5	GRPO (Group Relative Policy Optimization)	34
7.5.1	KL in the Loss vs. KL in the Reward	35
7.6	RLVR (Reinforcement Learning from Verifiable Rewards)	35
7.7	Comparison of RL Methods	35
7.8	Other Alignment Methods	35
7.8.1	K1 vs. K3 KL Estimators	36
7.9	Exercises	36
III	Beyond Transformers	37
8	State Space Models and Alternative Architectures	39
8.1	From RNNs to Structured State Spaces	39
8.2	S4: Structured State Space Sequence Model	39

8.3	H3: Leveraging SSMs for Language	40
8.4	Mamba: S6 Selective SSM	40
8.5	Mamba-2: SSD Reformulation	40
8.6	Mamba-3: Complex States and MIMO	41
8.7	Hybrid Architectures	41
8.8	MoE-SSM Hybrids	41
8.9	Comparison with Transformers	41
8.10	Exercises	42
IV	Inference	43
9	Autoregressive Inference	45
9.1	Decoding Strategies	45
9.2	Temperature Sampling	45
9.3	The KV Cache	46
9.4	Parallel Decoding	46
9.5	Exercises	46
10	Inference Optimisation	47
10.1	Quantisation	47
10.1.1	GPTQ	47
10.1.2	AWQ	47
10.1.3	GGUF and NF4	47
10.1.4	FP8 Inference (H100)	47
10.2	Speculative Decoding	48
10.3	Flash Attention	48
10.4	Continuous Batching and PagedAttention	48
10.5	Exercises	48
11	Long-Context Techniques	51
11.1	Position Interpolation (PI)	51
11.2	NTK-Aware Scaling	51
11.3	YaRN (Yet another RoPE extension)	51
11.4	Dynamic NTK	52
11.5	LongRoPE	52
11.6	LongRoPE2	52
11.7	AlphaRoPE	53
11.8	CoPE (Contextual Position Encoding)	53
11.9	RoPE Out-of-Distribution Theory	53
11.10	Comparison	53
11.11	Exercises	54
V	Engineering	55
12	GPU Architecture for LLMs	57
12.1	GPU Memory Hierarchy	57
12.2	Compute vs. Memory Bound Analysis	57
12.3	Tensor Cores and Mixed Precision	57
12.3.1	Hopper-Specific Features	57
12.4	Parallelism Strategies	58

12.4.1 Interconnects	58
12.5 Kernel Fusion	58
12.6 Exercises	58
13 Software Engineering for LLMs	59
13.1 Prompt Engineering Patterns	59
13.2 Retrieval-Augmented Generation (RAG)	59
13.2.1 Advanced RAG Patterns	60
13.3 Agent Architectures	60
13.4 Testing and Evaluation	60
13.5 Production Deployment Patterns	60
13.6 Exercises	60
VI Practice	61
14 Libraries and Frameworks	63
14.1 HuggingFace Transformers	63
14.2 vLLM	63
14.3 TensorRT-LLM	63
14.4 llama.cpp	64
14.5 LangChain	64
14.6 LlamaIndex	64
14.7 Comparison Table	64
14.8 Exercises	65
15 Evaluation and Advanced Topics	67
15.1 Benchmark Suites	67
15.2 Hallucination and Calibration	67
15.2.1 Hallucination Taxonomies	67
15.3 Safety Evaluation	68
15.4 Outlook: The Future of LLM Engineering	68
15.5 Final Project	68

Part I

Foundations

Chapter 1

Information-Theoretic Foundations

“The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point.”

Claude Shannon, 1948

This chapter develops the information-theoretic lens through which large language models (LLMs) are best understood. Language modelling is, at its core, a problem of efficient coding: a good language model assigns high probability to the next token, minimising the number of bits needed to encode the message.

1.1 Entropy and Language

Definition 1.1 (Entropy). For a discrete random variable X with distribution p , the Shannon entropy is

$$H(X) = - \sum_x p(x) \log_2 p(x) \quad (\text{bits}). \quad (1.1)$$

Entropy measures the *minimum* number of bits required to encode X on average.

In natural language, the per-token entropy of English has been estimated at roughly 1.3 bits/character (Shannon’s game), though modern LLMs achieve cross-entropies well below 1 bit/token on domain-specific corpora.

Definition 1.2 (Cross-Entropy). Given a true distribution p and a model q , the cross-entropy is

$$H(p, q) = - \sum_x p(x) \log_2 q(x) = H(p) + D_{\text{KL}}(p \parallel q). \quad (1.2)$$

Minimising cross-entropy is equivalent to minimising KL divergence from the true distribution.

This is precisely the objective minimised during language model training. For a sequence $\mathbf{x} = (x_1, \dots, x_T)$, the per-token cross-entropy loss is

$$\mathcal{L}_{\text{NTP}} = - \frac{1}{T} \sum_{t=1}^T \log_2 p_{\theta}(x_t \mid x_{<t}), \quad (1.3)$$

known as *next-token prediction* (NTP).

Theorem 1.1 (Shannon’s Source Coding Theorem). For a stationary ergodic source with entropy rate H , the minimum expected code length per symbol achievable by any uniquely decodable code satisfies

$$H \leq L^* < H + 1. \quad (1.4)$$

This theorem provides the fundamental limit: no language model can achieve a cross-entropy lower than the true entropy of the language. In practice, LLMs approach but never reach this limit, and the gap is precisely the KL divergence.

1.2 Kullback–Leibler Divergence

Definition 1.3 (KL Divergence).

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0, \quad (1.5)$$

with equality iff $p = q$ almost everywhere.

KL divergence appears throughout LLM theory:

- **Training:** minimising cross-entropy is minimising KL divergence.
- **Distillation:** student models are trained to minimise $D_{\text{KL}}(p_{\text{teacher}} \parallel p_{\text{student}})$.
- **RLHF:** the KL penalty term $\beta D_{\text{KL}}(\pi_{\text{RL}} \parallel \pi_{\text{SFT}})$ prevents the policy from diverging too far from the supervised model.
- **Beam search:** top- k and top- p sampling truncate the distribution, inducing controlled KL divergence.

Theorem 1.2 (Gibbs’ Inequality). For any two probability distributions p, q over the same discrete space,

$$D_{\text{KL}}(p \parallel q) \geq 0, \quad (1.6)$$

with equality if and only if $p = q$ almost everywhere.

Proof. By Jensen’s inequality applied to the convex function $-\log$:

$$D_{\text{KL}}(p \parallel q) = - \sum_x p(x) \log \frac{q(x)}{p(x)} \geq - \log \sum_x p(x) \frac{q(x)}{p(x)} = - \log 1 = 0. \quad (1.7)$$

□

1.3 Mutual Information in Attention

Definition 1.4 (Mutual Information).

$$I(X; Y) = D_{\text{KL}}(p_{(X,Y)} \parallel p_X \otimes p_Y) = H(X) - H(X \mid Y). \quad (1.8)$$

Mutual information measures the reduction in uncertainty about X given Y .

The attention mechanism computes a weighted sum of value vectors, where weights are given by a softmax over dot products:

$$\alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_{j'} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k})}. \quad (1.9)$$

This can be interpreted as a (normalised) pointwise mutual information between positions i and j in the latent space. The attention distribution $p(j \mid i)$ maximises the mutual information between the query position and the context positions under a soft constraint on the number of attended positions.

Theorem 1.3 (Data Processing Inequality). If $X \rightarrow Y \rightarrow Z$ forms a Markov chain, then

$$I(X; Y) \geq I(X; Z). \quad (1.10)$$

In transformer layers, this implies that information about the input is monotonically non-increasing as it propagates through layers, though the *task-relevant* information may increase through the information bottleneck.

1.4 Rate-Distortion Theory and Quantisation

Quantisation of LLM weights can be viewed through rate-distortion theory.

Definition 1.5 (Rate-Distortion Function). For a source X and distortion measure d , the rate-distortion function is

$$R(D) = \min_{p(\hat{x}|x): \mathbb{E}[d(X, \hat{X})] \leq D} I(X; \hat{X}). \quad (1.11)$$

$R(D)$ gives the minimum number of bits per symbol needed to reconstruct the source with average distortion at most D .

Post-training quantisation (PTQ) maps each weight $w \in \mathbb{R}$ to $\hat{w} \in \{0, \dots, 2^b - 1\}$, typically with a scale and zero-point:

$$\hat{w} = \text{clamp} \left(\left\lfloor \frac{w}{\Delta} \right\rfloor + z, 0, 2^b - 1 \right). \quad (1.12)$$

The distortion $d(w, \hat{w}) = (w - \hat{w})^2$ induces a degradation D in model perplexity. Rate-distortion theory tells us that for a given bit-width b , there is a fundamental trade-off between compression rate and model quality.

1.5 The Information Bottleneck and Hidden Representations

The information bottleneck (IB) principle (Tishby et al., 1999) provides a framework for understanding transformer hidden states. An LLM layer computes a representation Z that should be:

- *Sufficient*: $I(Z; Y) \geq I(X; Y)$ (preserve task-relevant information).
- *Minimal*: minimise $I(Z; X)$ (compress away irrelevant details).

The IB Lagrangian is

$$\mathcal{L}_{\text{IB}} = I(Z; X) - \beta I(Z; Y). \quad (1.13)$$

In transformers, each layer's hidden state h_ℓ can be seen as solving a successive IB trade-off, progressively filtering out task-irrelevant information while preserving predictive signal. This perspective explains why later layers are more task-specific and why pruning early layers often preserves performance.

1.6 The Entropy of Natural Language

Estimating the entropy of natural language is a longstanding problem:

- **Shannon's game (1951)**: humans guess the next letter; entropy ≈ 1.3 bits/character.
- **Cross-entropy of n-gram models**: ~ 1.75 bits/char (5-gram on Brown corpus).
- **Neural language models**: GPT-2 achieves ~ 1.0 bits/char on WikiText-2.
- **LLMs**: GPT-4 achieves ~ 0.7 bits/char on carefully curated benchmarks.

The gap between model cross-entropy and the true entropy of the language ($H_{\text{true}} \approx 0.6$ – 1.0 bits/char) represents the irreducible loss due to fundamental unpredictability in natural language.

1.7 Exercises

1. Prove that $H(p, q) \geq H(p)$ with equality iff $p = q$.
2. Show that the cross-entropy loss for next-token prediction is an unbiased estimator of $H(p, p_\theta)$ in the limit of infinite data.

3. For a quantised weight with b bits, derive the expected MSE under uniform quantisation of a $\mathcal{N}(0, \sigma^2)$ distribution.
4. Compute $R(D)$ for a Gaussian source with squared-error distortion.
5. Explain why the KL penalty in RLHF prevents mode collapse.
6. Prove the data processing inequality for Markov chains.
7. Show that the IB Lagrangian can be rewritten as $\mathcal{L}_{\text{IB}} = H(Y | Z) + \beta I(Z; X)$.

Chapter 2

Computational Theory of Language Models

“Computation is the process of transforming information according to well-defined rules.”

Alan Turing

This chapter examines LLMs through the lens of theoretical computer science: formal languages, computational complexity, and expressiveness.

2.1 The Autoregressive Paradigm

An autoregressive language model defines a distribution over sequences:

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}). \quad (2.1)$$

This is a *generative* process: each token is sampled conditioned on all previous tokens. The computational cost is fundamentally sequential—generating token t requires having generated all $t - 1$ prior tokens.

Theorem 2.1 (Sequential Lower Bound). Any exact autoregressive generation of a sequence of length T requires at least T sequential steps, regardless of parallel hardware.

Proof. By induction. The distribution of x_t depends on $x_{<t}$. Without knowledge of x_{t-1} , the marginal $p(x_t \mid x_{<t-1})$ differs from $p(x_t \mid x_{<t})$ unless the model is provably Markov of order $t - 2$, which is not the case for general LLMs. $\square$

This bound motivates non-autoregressive and speculative decoding techniques, which trade exactness for parallelism.

2.2 Computational Complexity of Attention

Let n be the sequence length and d the hidden dimension.

- **Self-attention:** $O(n^2d)$ time, $O(n^2)$ memory for the attention matrix.
- **Feed-forward:** $O(nd^2)$ time, $O(d^2)$ memory.
- **Autoregressive generation (per step):** $O(nd)$ for attention with KV cache (incremental), $O(d^2)$ for FFN.

Theorem 2.2 (Context Window as Memory). The attention mechanism implements a content-addressable memory of size $O(nd)$. Retrieving information from a context of length n requires $O(n)$ work, matching the lower bound for associative memory.

Theorem 2.3 (Quadratic Lower Bound for Exact Attention). Computing the full attention matrix $A = \text{softmax}(QK^\top/\sqrt{d_k})$ requires $\Omega(n^2)$ operations in the worst case, as every pair of positions (i, j) must contribute to the output.

This quadratic barrier is the primary motivation for sparse attention patterns, linear attention, and SSM alternatives.

2.3 Expressiveness: What Can Transformers Compute?

Theorem 2.4 (Universal Approximation for Sequence-to-Sequence). A transformer with sufficient width and depth can approximate any continuous sequence-to-sequence function on a compact domain, provided it has access to positional encodings.

Theorem 2.5 (Turing Completeness). Transformers with rational-valued activations and CoT (chain-of-thought) prompting are Turing-complete. A transformer with $O(\log n)$ layers and polynomial width can simulate a Turing machine for n steps.

Corollary 2.1 (CoT Increases Expressiveness). There exist problems (e.g., evaluating Boolean formulas, solving linear equations) that cannot be solved by a constant-depth transformer without CoT but can be solved by the same transformer with $O(n)$ chain-of-thought steps.

2.4 Circuit Complexity and TC0

The class TC^0 consists of problems solvable by constant-depth, polynomial-size circuits with threshold (majority) gates. Transformers with no chain-of-thought are believed to be limited to problems in TC^0 :

- TC^0 includes: parity, addition, multiplication by constants, sorting.
- TC^0 does NOT include: multiplication of two n -bit numbers (requires NC^1), matrix multiplication, or simulating a Turing machine for n steps.
- With CoT, transformers can simulate P (polynomial time) by using the token sequence as a tape.

Theorem 2.6 (Transformer Hierarchy). For a transformer with depth L , hidden dimension d , and precision p :

- Without CoT: computable functions lie in TC^0 (for $L = O(1)$, $d = \text{poly}(n)$).
- With $O(\log n)$ CoT steps: NC^1 .
- With $\text{poly}(n)$ CoT steps: P.

2.5 RASP: A Programming Language for Transformers

RASP (Weiss et al., 2021) is a restricted programming language that captures the computational primitives of transformers:

- **Selector operations:** attend from position i to j based on a predicate.
- **Aggregate:** compute a softmax-weighted average of values.
- **Sequence operations:** map, filter, reduce over positions.

Examples of RASP programs and their transformer realisations:

- **Histogram:** count token frequencies in $O(1)$ layers.
- **Flip:** reverse a sequence in $O(1)$ layers.

- **dyck-1**: recognise balanced parentheses in $O(\log n)$ layers.
- **Prefix sum**: compute running sums in $O(1)$ layers.

2.6 Injective LLMs and Input Reconstruction

A recent line of work (arXiv:2510.15511) asks: can LLM-generated outputs be inverted to reconstruct the original input?

Definition 2.1 (Injective Language Model). An LLM p_θ is injective if the mapping from input sequences to output distributions is one-to-one: different inputs always produce distinguishable output distributions.

The SIPIT (Sampling-based Input Reconstruction) algorithm reconstructs the input prefix from generated tokens:

Algorithm 2.1 (SIPIT). Given output tokens $y_1, \dots, y_m$, reconstruct input $x_1, \dots, x_n$ by:

1. Sampling multiple continuations from the model.
2. Identifying input tokens that maximise the likelihood of observed outputs.
3. Applying beam search over the input space.

Empirically, SIPIT reconstructs up to 60% of input tokens exactly for GPT-2, showing that LLMs encode substantial information about their inputs in their outputs.

2.7 Formal Language Hierarchy

LLMs correspond to different levels of the Chomsky hierarchy:

- **Regular languages**: exactly expressible by bounded-length n -gram models (Markovian). A transformer without positional encodings is bounded to regular languages.
- **Context-free languages**: require stack-like memory; transformers with positional encodings can recognise some but not all CFLs. Dyck languages of depth k require $\Omega(\log n)$ layers.
- **Context-sensitive languages**: require $O(n)$ memory; transformers with sufficient layers can recognise some CSLs via chain-of-thought.
- **Recursively enumerable**: Turing-complete with CoT.

Theorem 2.7 (Separation by Depth). For any k , there exists a formal language L_k recognisable by a transformer of depth $k+1$ but not by any transformer of depth k (with bounded precision).

2.8 The Scaling Hypothesis

Theorem 2.8 (Scaling Laws). For a transformer trained on D tokens with N parameters, the cross-entropy loss scales as

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2.2)$$

where E is the irreducible entropy of the data, A, B, α, β are constants, and $\alpha \approx \beta \approx 0.5$ for typical architectures.

The optimal allocation under a fixed compute budget C satisfies $N_{\text{opt}} \propto C^{0.5}$ and $D_{\text{opt}} \propto C^{0.5}$ (Chinchilla scaling), implying model size and data should be scaled equally.

Corollary 2.2 (Overtraining). Training a model beyond the Chinchilla-optimal point (more tokens than optimal) yields diminishing returns: doubling tokens gives at most $2^{-\beta} \approx 70\%$ of the loss reduction of optimally allocated compute.

2.9 Embedding Condensation

A recent finding (arXiv:2602.00217) shows that smaller models can “condense” the embedding geometry of larger models:

Theorem 2.9 (Embedding Condensation). Training a smaller student model with a *dispersion loss* $\mathcal{L}_{\text{disp}} = \|G_S - G_T\|_F^2$, where G_S and G_T are Gram matrices of hidden representations, transfers the geometric structure of the larger model’s embedding space.

This suggests that the embedding geometry—the relative arrangement of token representations—carries substantial information that can be transferred across model scales.

2.10 Exercises

1. Prove that exact autoregressive generation is in P but each step depends on all previous outputs.
2. Show that the memory complexity of self-attention is $\Omega(n^2)$ for exact computation.
3. Argue why a transformer without positional encodings cannot recognise the language $a^n b^n$.
4. Derive the Chinchilla-optimal allocation from the scaling law equation.
5. Explain why chain-of-thought can increase effective depth.
6. Write a RASP program that computes the prefix sum of a binary sequence.
7. Show that $\text{TC}^0 \subseteq \text{NC}^1$ and argue why transformer limitations follow.

Part II

Architecture and Training

Chapter 3

Transformer Architecture

“Attention is all you need.”

Vaswani et al., 2017

3.1 The Transformer Block

Each transformer layer consists of two sub-layers:

1. Multi-head self-attention (MHSA)
2. Feed-forward network (FFN), typically a gated MLP

Each sub-layer employs a residual connection and layer normalisation:

$$h' = \text{LayerNorm}(h + \text{SubLayer}(h)). \quad (3.1)$$

3.1.1 Pre-Norm vs. Post-Norm

The original transformer (Vaswani et al.) applied layer norm *after* the residual addition (post-norm):

$$h' = \text{LayerNorm}(h + \text{SubLayer}(h)). \quad (3.2)$$

Modern LLMs (GPT-3, Llama, Mistral) apply layer norm *before* each sub-layer (pre-norm):

$$h' = h + \text{SubLayer}(\text{LayerNorm}(h)). \quad (3.3)$$

Pre-norm provides more stable gradients at initialization, enabling training of deeper models without warmup instability.

3.1.2 RMSNorm

Llama and many subsequent models replace LayerNorm with RMSNorm:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}} \cdot \gamma, \quad (3.4)$$

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta. \quad (3.5)$$

RMSNorm removes the mean-centering step, reducing computation by $O(d)$ per layer with negligible quality loss.

3.2 Masked Multi-Head Attention

In autoregressive decoding, each position can only attend to itself and earlier positions:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad (3.6)$$

where M is a causal mask: $M_{ij} = 0$ for $i \geq j$, $-\infty$ otherwise.

Multi-head attention splits the computation into h heads:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (3.7)$$

$$\text{MHSA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O. \quad (3.8)$$

3.3 Grouped Query Attention (GQA)

Modern LLMs (Llama 2/3, Mistral) use GQA where multiple query heads share a key-value head:

$$n_{kv} = \frac{n_q}{g}, \quad g \geq 1. \quad (3.9)$$

GQA reduces KV cache memory by a factor of g with minimal quality loss. It interpolates between multi-head attention ($g = 1$) and multi-query attention ($g = n_q$).

Theorem 3.1 (GQA Memory Savings). For a model with L layers, n_{kv} KV heads, head dimension d_h , sequence length t , and precision p bytes, the KV cache memory is:

$$M_{KV} = 2 \cdot L \cdot n_{kv} \cdot t \cdot d_h \cdot p. \quad (3.10)$$

With GQA ratio g , this becomes M_{KV}/g compared to full MHA.

3.4 Feed-Forward Networks

The SwiGLU activation has become standard:

$$\text{FFN}(x) = (W_1x) \odot \sigma_{\text{SiLU}}(W_2x) \cdot W_3, \quad (3.11)$$

where $\sigma_{\text{SiLU}}(z) = z \cdot \sigma(z)$ is the SiLU/Swish activation, σ is the logistic sigmoid, and $\odot$ denotes element-wise multiplication.

Other GLU variants:

- **ReGLU**: $(W_1x) \odot \max(0, W_2x) \cdot W_3$.
- **GeGLU**: $(W_1x) \odot \text{GELU}(W_2x) \cdot W_3$.
- **SwiGLU** (default in Llama, Mistral, GPT-4).

For a hidden dimension d and intermediate dimension $d_{ff} = \frac{8}{3}d$, SwiGLU has three weight matrices vs. two in the standard ReLU FFN, adding 33% more parameters.

3.5 RoPE: Rotary Positional Encodings

RoPE encodes position via a rotation matrix:

$$R_{\Theta, m} = \bigoplus_{j=1}^{d/2} \begin{pmatrix} \cos m\theta_j & -\sin m\theta_j \\ \sin m\theta_j & \cos m\theta_j \end{pmatrix}, \quad (3.12)$$

where $\theta_j = 10000^{-2j/d}$.

The dot product $q_m^\top k_n$ in RoPE depends only on the relative position $m - n$:

$$q_m^\top k_n = (R_{\Theta, m} x_q)^\top (R_{\Theta, n} x_k) = x_q^\top R_{\Theta, m-n} x_k. \quad (3.13)$$

This relative-position property enables length generalisation. The rotating frequency θ_j determines how quickly position information decays: lower dimensions rotate faster (capturing local positions), higher dimensions rotate slower (capturing long-range positions).

3.6 Attention Variants

- **ALiBi (Press et al., 2022)**: adds a linear bias proportional to distance: $\text{softmax}(QK^\top / \sqrt{d} + m \cdot B)$ where $B_{ij} = -|i - j|$. No positional embeddings needed; extrapolates better to long sequences.
- **Attention with Linear Biases (xPos)**: combines RoPE with exponential decay for better length generalisation.
- **Sparse attention**: only attend to local windows + global tokens (BigBird, Longformer).

3.7 Exercises

1. Derive the FLOPs count per transformer layer.
2. Show that GQA with g groups reduces KV cache memory by a factor of g .
3. Prove that RoPE makes attention depend only on relative position.
4. Implement a single masked attention head in pseudocode.
5. Compare pre-norm and post-norm in terms of gradient signal propagation.
6. Compute the parameter count of a SwiGLU FFN vs. a standard ReLU FFN for the same d_{ff} .

Chapter 4

Tokenization

“A tokenizer is a software stack, not a modelling choice.”

Andrej Karpathy, 2023

Tokenization converts raw text into a sequence of integers (token IDs) that the model processes. Despite being a preprocessing step, tokenization choices profoundly affect model behaviour, vocabulary efficiency, and multilingual capability.

4.1 Byte-Pair Encoding (BPE)

BPE (Gage, 1994; Sennrich et al., 2016) is the dominant tokenization algorithm, used by GPT-2/3/4, Llama, and most modern LLMs.

Algorithm 4.1 (BPE Tokenizer Training). 1. Start with a vocabulary of individual bytes (or characters).
2. Count all adjacent pairs of tokens in the corpus.
3. Merge the most frequent pair into a new token.
4. Repeat step 2–3 until the desired vocabulary size is reached.

Definition 4.1 (BPE Merge). A BPE merge operation replaces every occurrence of the adjacent pair (a, b) in the corpus with the new symbol ab . The merge is *greedy*: once merged, the pair becomes an atomic unit for all subsequent merges.

The BPE tokenizer for GPT-2 operates on bytes (Byte-Level BPE), ensuring that any Unicode string can be tokenized without unknown tokens. Before BPE, most tokenizers operated on characters or word-level, requiring large vocabularies for multilingual coverage.

Example 4.1 (BPE Merge Sequence). For the corpus {“low”, “lower”, “lowest”}:

1. Initial: {l, o, w, e, r, s, t}
2. Merges: l+o → lo, w+e → we, lo+w → low, we+r → wer, low+e → lowe, lowe+r → lower, low+e → lowe, low+s → lows, lowe+s → lowes, lowes+t → lowest

The final vocabulary size depends on the merge count limit.

4.2 SentencePiece

SentencePiece (Kudo & Richardson, 2018) treats tokenization as a segmentation problem without needing pre-tokenized words (whitespace is handled as a regular symbol `_`). It supports two algorithms:

4.2.1 BPE Mode

Same merge-based algorithm as BPE, but operating on the raw Unicode byte stream rather than pre-tokenized words. Whitespace is preserved as the underscore character.

4.2.2 Unigram Language Model Mode

Algorithm 4.2 (Unigram LM Tokenization). 1. Start with a large seed vocabulary (all characters + frequent substrings).
 2. Define $p(x) = \sum_{s \in S(x)} p(s)$, where $S(x)$ is the set of segmentations of x .
 3. Optimise vocabulary by removing tokens that reduce likelihood the least.
 4. Repeat until the desired vocabulary size is reached.

The unigram model defines the probability of a tokenisation as:

$$p(\mathbf{x}) = \prod_{i=1}^n p(t_i), \quad \text{where } t_i \text{ are the tokens.} \quad (4.1)$$

Training maximises the marginal likelihood $p(\text{corpus}) = \prod_{x \in \text{corpus}} \sum_{S \in \text{seg}(x)} \prod_{t \in S} p(t)$, which is computed efficiently via the forward-backward algorithm.

4.3 tiktoken

tiktoken is OpenAI’s fast BPE tokenizer, implemented in Rust with Python bindings. It provides several encoding schemes:

- **r50k_base**: GPT-3 (50K vocabulary, code and text).
- **cl100k_base**: GPT-3.5 and GPT-4 (100K vocabulary, extended multilingual and code support).
- **o200k_base**: GPT-4o (200K vocabulary, significantly expanded multilingual and emoji coverage).

Theorem 4.1 (Vocab Size Evolution). The progression of tokenizer vocabulary sizes reflects the need for:

- **50K (GPT-3, 2020)**: sufficient for English-dominant text.
- **100K (GPT-3.5/4, 2023)**: better multilingual coverage and code efficiency.
- **200K (GPT-4o, 2024)**: broad multilingual parity and reduced token/byte ratio for CJK languages.

4.4 WordPiece

WordPiece, used in BERT and its variants, differs from BPE in its merge criterion:

Algorithm 4.3 (WordPiece Merge Criterion). Instead of merging the most frequent pair, WordPiece merges the pair (a, b) that maximises the likelihood increase:

$$\Delta \mathcal{L} = \log \frac{p(b \mid a)}{p(b)} = \log \frac{\text{count}(a, b)}{\text{count}(a)\text{count}(b)}. \quad (4.2)$$

This prefers pairs whose co-occurrence is higher than expected by chance.

4.5 Vocabulary Size Trade-offs

Vocab Size	Advantages	Disadvantages	Examples
32K (small)	Small embedding matrix, fast	High OOV rate, poor multiling.	Llama 1/2
50K (medium)	Balanced for English	CJK tokenisation inefficient	GPT-3, GPT-2
100K (large)	Better multilingual	Larger embedding layer	GPT-3.5/4, Llama 3
128K (large)	Good coverage	25%+ of params in embeddings	Llama 3
200K (XL)	Minimal OOV	Very large embedding	GPT-4o, o200k_base

For a vocabulary size V and hidden dimension d , the embedding matrix has $V \cdot d$ parameters. At $V = 128K$ and $d = 8192$, the embedding accounts for over 1B parameters—a significant fraction of an 8B model.

4.6 Multilingual Tokenization and CJK

Languages with large character sets (Chinese, Japanese, Korean) expose weaknesses in BPE:

- English text averages ~ 1.3 tokens/word.
- Chinese text averages ~ 2.5 – 3.5 tokens/character due to UTF-8 byte decomposition.
- A Chinese character (e.g., the character for “middle”) is split into multiple bytes before BPE merging.
- Llama 3’s 128K vocabulary was specifically designed to improve CJK efficiency.
- GPT-4o’s 200K vocabulary dramatically reduces CJK token/byte ratios.

Definition 4.2 (Token-to-Byte Ratio). The token-to-byte ratio measures tokenization efficiency:

$$\rho = \frac{\text{number of tokens}}{\text{number of bytes}}. \quad (4.3)$$

Lower ρ means more efficient encoding. For English, $\rho \approx 0.25$ (GPT-4), for Chinese, $\rho \approx 0.6$ – 1.0 depending on vocabulary size.

4.7 Digit Tokenization and Arithmetic Failures

BPE tokenization of numbers causes systematic arithmetic failures:

- The number “15213” may be tokenized as [“15”, “213”] or [“1”, “5213”] depending on context.
- Arithmetic operations that should be consistent across digit representations become non-compositional.
- The model must learn arithmetic independently for each digit substring, rather than leveraging place-value structure.
- Switching to per-digit tokenization (“1”, “5”, “2”, “1”, “3”) resolves many arithmetic failures but increases sequence length.

This is a known limitation: BPE merges frequent digit pairs (like “00” or “99”), destroying the place-value structure that makes arithmetic systematic.

4.8 Special Tokens

Tokenizers reserve special tokens for structural control:

- `<|begin_of_text|>` / `<|end_of_text|>`: sequence boundaries.

- `<|pad|>`: padding for batch processing.
- `<|unk|>`: unknown token (rare in byte-level tokenizers).
- `<|im_start|>`, `<|im_end|>`: chat message delimiters.
- `<|tool_call|>`: structured tool invocation markers.
- `<|reserved|>`: reserved for task-specific tokens (e.g., special tokens for function calling in GPT-4).

The design of special tokens has become increasingly important for structured output generation and agentic workflows.

4.9 Formal Semantics of BPE

Recent work (arXiv:2309.08715) formalises BPE tokenization semantics:

Theorem 4.2 (BPE Determinism). Byte-level BPE with a fixed merge table defines a deterministic tokenization of any string, provided no merge creates a token absent from the vocabulary.

Theorem 4.3 (BPE Non-Monotonicity). BPE tokenization is not monotonic: adding a merge can change the tokenization of strings that did not contain the merged pair, because merges interact through shared prefixes and suffixes.

This non-monotonicity means that tokenization decisions are globally interdependent—merges cannot be applied independently.

4.10 The Tokenizer Wars (2023–2024)

The importance of tokenizer design became a debated topic:

- Some argued that tokenizer choice is as important as model architecture (and that GPT-4o’s quality improvements partly stem from its better tokenizer).
- Others countered that scaling the vocabulary is a straightforward engineering optimization, not a research breakthrough.
- The release of GPT-4o’s `o200k_base` encoding showed clear improvements in multilingual token/byte ratios, suggesting that prior tokenizers were suboptimal for non-English languages.

The consensus emerging by 2025: tokenizer quality matters, especially for multilingual performance and code generation, but it is a necessary baseline consideration rather than a differentiating research contribution.

4.11 Exercises

1. Implement BPE merge training on a small corpus of 100 sentences.
2. Compare the tokenization efficiency of GPT-2’s 50K tokenizer vs. GPT-4o’s 200K tokenizer on a Chinese Wikipedia article.
3. Show that BPE digit tokenization makes arithmetic non-compositional.
4. Compute the embedding parameter count for $V = 128K$, $d = 8192$.
5. Write pseudocode for the SentencePiece unigram LM training loop.
6. Explain why WordPiece’s merge criterion differs from BPE’s.

Chapter 5

Mixture of Experts

“Sparsity is the ultimate regularizer.”

Anonymous

Mixture-of-Experts (MoE) layers replace dense FFN layers with multiple “expert” sub-networks, routing each token to a subset of experts. This decouples model capacity (total parameters) from computational cost (FLOPs per token).

5.1 Top- k Routing

Definition 5.1 (MoE Layer). An MoE layer consists of E expert networks $f_1, \dots, f_E$ and a router g :

$$y = \sum_{i=1}^E g(x)_i \cdot f_i(x), \quad (5.1)$$

where $g(x) \in \mathbb{R}^E$ is a sparse gating vector.

Definition 5.2 (Top- k Routing). The router computes a score for each expert and activates only the top k :

$$g(x) = \text{softmax}(\text{TopK}(W_g x + \epsilon, k)), \quad (5.2)$$

where $\text{TopK}(v, k)_i = v_i$ if v_i is in the k largest values, and $-\infty$ otherwise. Typically $k = 2$ for a balance of capacity and computational efficiency.

The computational cost of an MoE layer with E experts, top- k routing, and hidden dimension d is k/E of the cost of a dense layer with $E \cdot d$ hidden units, while retaining $E \times$ the parameters.

5.2 Token Choice vs. Expert Choice Routing

- **Token Choice Routing (TCR)**: each token selects its top- k experts. Simple and widely used (Switch Transformer, Mixtral). May cause load imbalance.
- **Expert Choice Routing (ECR)**: each expert selects its top- k tokens. Guarantees perfect load balance. Used in M6-T and later V-MoE (Vision MoE).

ECR addresses a fundamental flaw of TCR: under TCR, tokens may all select the same expert (catastrophic load imbalance), while other experts receive no tokens.

Theorem 5.1 (ECR Load Balance Guarantee). With Expert Choice Routing and $T = k \cdot E$ tokens, each expert processes exactly k tokens per layer, providing deterministic load balance.

5.3 Load Balancing Loss

To prevent the router from collapsing to a few experts, an auxiliary load balancing loss is added:

$$\mathcal{L}_{\text{bal}} = \alpha \cdot E \cdot \sum_{i=1}^E \left(\frac{1}{T} \sum_{t=1}^T \mathbb{1}\{\text{token } t \text{ routed to expert } i\} \right) \cdot \left(\frac{1}{T} \sum_{t=1}^T g(x_t)_i \right). \quad (5.3)$$

This minimises the product of the fraction of tokens assigned to each expert and the average routing probability. When both are uniform ($1/E$), the loss is minimal.

Remark 5.1. The load balancing loss trades off between routing quality (which may favour imbalanced routing) and computational efficiency (which requires balanced routing). The coefficient α controls this trade-off, typically set to 0.01.

5.4 Fine-Grained Experts: DeepSeekMoE

DeepSeekMoE (DeepSeek, 2024) introduces fine-grained expert splitting:

- **Shared experts:** E_s experts are always activated for every token.
- **Routed experts:** each token activates k of E_r routed experts via top- k .

The output is:

$$y = \sum_{i=1}^{E_s} f_i^{(\text{shared})}(x) + \sum_{i=1}^{E_r} g(x)_i \cdot f_i^{(\text{routed})}(x). \quad (5.4)$$

Theorem 5.2 (Fine-Grained Expert Advantage). Splitting a single expert into multiple smaller experts and routing to more of them (e.g., $E = 8, k = 2$ vs. $E = 64, k = 8$) improves the knowledge-capacity trade-off: each token accesses more specialised knowledge for the same computational cost.

DeepSeek-V2 uses $E_s = 2$ shared experts and $E_r = 160$ routed experts with $k = 6$, achieving 1/th the inference cost of a comparable dense model.

5.5 Expert Parallelism

Definition 5.3 (Expert Parallelism). Each expert is placed on a specific GPU (or GPU group). Tokens are routed to the GPU hosting their selected expert via all-to-all communication:

$$T_{\text{comm}} = T_{\text{all-to-all}}(B \cdot d \cdot E), \quad (5.5)$$

where B is the batch size and d the hidden dimension.

Expert parallelism introduces communication overhead: tokens must be sent to the GPU holding their expert, processed, and returned. This can become a bottleneck for small batch sizes.

5.6 Grouped GEMM for MoE

Standard MoE implementations process experts independently, launching E separate matrix multiplications. Grouped GEMM fuses these into one kernel:

- Tokens for all experts are packed into a contiguous input tensor.
- A single grouped GEMM kernel dispatches each token to its expert’s weight matrix.
- Eliminates $E - 1$ kernel launch overheads.
- Requires expert weight matrices arranged as [expert, hidden, intermediate].

Theorem 5.3 (Grouped GEMM Speedup). For E experts and batch B , grouped GEMM achieves speedup:

$$S = \frac{E \cdot (T_{\text{kernel}} + T_{\text{launch}})}{T_{\text{kernel}} + E \cdot T_{\text{launch}}} \approx E \quad \text{for small batches.} \quad (5.6)$$

5.7 PathMoE

PathMoE (arXiv:2604.17837) introduces *block-shared routers* to improve routing consistency:

- Each MoE layer has a shared router g_{shared} and per-expert routers g_i .
- The routing decision decomposes into a *control signal* (which expert type) and a *content channel* (which specific knowledge).
- Output: $y = g_{\text{shared}}(x) \cdot \sum_i g_i(x) \cdot f_i(x)$.

Theorem 5.4 (Routing Decomposition). PathMoE achieves 79% routing consistency across similar inputs, compared to $\sim 30\%$ for standard top- k routing. The control signal captures the broad skill type, while the content channel specialises within that skill.

5.8 Polysemantic Experts vs. Monosemantic Paths

An important finding about MoE interpretability:

- **Polysemantic experts:** individual experts activate for multiple unrelated concepts (e.g., the same expert handles both code syntax and mathematical notation).
- **Monosemantic paths:** the *combination* of which experts are activated forms a sparse code that is more interpretable.

Theorem 5.5 (Expert Superposition). In large MoE models, individual experts are polysemantic (they activate for multiple unrelated inputs), but the routing vector $g(x) \in \mathbb{R}^E$ acts as a sparse code that disentangles features. The effective dimensionality of the representation grows with E even though each token activates only k experts.

5.9 Major MoE Models

Model	Experts	Top- k	Active params	Notable feature
Switch Transformer	2048	1	$\sim 7\text{B}$	First large-scale MoE LLM, instability at $k = 1$
GShard	2048	2	$\sim 7\text{B}$	Load balancing via capacity factor
Mixtral 8 $\times$ 7B	8	2	12.9B / 47B total	First popular open MoE
DeepSeek-V2	160 + 2 shared	6	21B / 236B total	Fine-grained + shared experts
Qwen1.5-MoE	64	8	2.7B / 14.3B total	k/E ratio = 1/8
DBRX	16	4	36B / 132B total	Fine-tuned for code

5.10 Exercises

1. Derive the FLOPs comparison between a dense FFN and an MoE FFN with E experts and top- k routing.
2. Show that Expert Choice Routing guarantees perfect load balance.
3. Compute the communication volume for expert parallelism with batch B , hidden d , and E experts.
4. Explain why the load balancing loss uses the product of assignment fraction and routing probability.
5. Compare the active-to-total parameter ratio across Mixtral, DeepSeek-V2, and DBRX.
6. Argue why routing consistency matters for model interpretability.

Chapter 6

Training and Scaling Laws

6.1 The Three-Stage Pipeline

Modern LLM training follows a three-stage process:

1. **Pre-training:** Next-token prediction on trillions of tokens. Objective:

$$\mathcal{L}_{\text{pretrain}} = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_t \log p_\theta(x_t \mid x_{<t}). \quad (6.1)$$

2. **Supervised fine-tuning (SFT):** Maximise likelihood of instruction–response pairs:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{D}_{\text{SFT}}|} \sum_{(x,y) \in \mathcal{D}_{\text{SFT}}} \log p_\theta(y \mid x). \quad (6.2)$$

3. **Reinforcement learning from human feedback (RLHF):** Optimise a reward model using the PPO objective with KL penalty:

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{(x,y) \sim \pi_\theta} [r(x,y)] - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}}). \quad (6.3)$$

6.2 Scaling Laws in Practice

The Kaplan et al. (2020) scaling laws establish power-law relationships:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} + L_\infty, \quad (6.4)$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} + L_\infty, \quad (6.5)$$

where $\alpha_N \approx 0.076$, $\alpha_D \approx 0.103$ for typical transformer architectures.

Definition 6.1 (Chinchilla Optimality). A model is Chinchilla-optimal when, for a given compute budget C , the loss $L(N, D)$ is minimised. The optimal allocation satisfies:

$$\frac{\partial L}{\partial N} \cdot \frac{dN}{dC} = \frac{\partial L}{\partial D} \cdot \frac{dD}{dC}. \quad (6.6)$$

For the standard scaling law, this yields $N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$.

Example 6.1 (Compute-Optimal Allocation). For a 70B model, Chinchilla-optimal training uses $\sim 1.4\text{T}$ tokens (20 tokens per parameter). Llama 3 70B was trained on 15T tokens, making it “overtrained” relative to Chinchilla—a deliberate choice to maximise downstream performance.

6.3 Learning Rate Schedules

Definition 6.2 (Cosine Decay).

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T} \pi \right) \right), \quad (6.7)$$

where T is the total number of training steps, $\eta_{\max}$ is the peak learning rate, and $\eta_{\min}$ is the final learning rate.

Modern practice (Llama 3, GPT-4):

- **Warmup:** linear increase from 0 to $\eta_{\max}$ over 2–5% of steps.
- **Cosine decay:** from $\eta_{\max}$ to $\eta_{\max} \cdot 0.1$ over the remaining steps.
- **Peak LR:** 3×10^{-4} for 7B, 1.5×10^{-4} for 70B (inverse sqrt scaling with model size).

6.4 Distributed Training Strategies

- **Data parallelism:** each GPU holds a full model replica, processes different micro-batches; gradients are all-reduced.
- **Tensor parallelism:** each GPU holds a slice of each layer; forward/backward passes require all-reduce on attention and FFN outputs.
- **Pipeline parallelism:** layers are partitioned across GPUs; micro-batches are pipelined through stages. Bubble overhead is $(P - 1)/(P \cdot M)$.
- **ZeRO optimisation:** partitions optimizer states, gradients, and parameters across GPUs, reducing memory per GPU.

6.4.1 Precision and Numerics

- **FP32:** master weights, loss scaling, normalisation.
- **BF16:** forward/backward pass, gradients. Larger exponent range than FP16 (no overflow for large gradients).
- **FP8 training (H100):** 8-bit floating point for matrix multiplications. Requires higher-precision accumulation and careful scaling.
- **Mixed precision:** FP32 master weights updated with FP16/BF16 gradients; loss scaling prevents underflow.

6.5 Exercises

1. Compute the total FLOPs for training a 7B-parameter model on 2T tokens.
2. Show that LoRA fine-tuning updates $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ with $r \ll d, k$.
3. Derive the memory savings of ZeRO Stage 3 over standard data parallelism.
4. Compute the Chinchilla-optimal data budget for a 13B model.
5. Explain why cosine decay outperforms step decay for LLM pre-training.

Chapter 7

Post-Training and Alignment

“Alignment is not a phase, it’s a discipline.”

Anonymous

Post-training converts a pre-trained base model into a helpful, harmless, and honest assistant. This chapter covers the major alignment techniques.

7.1 Supervised Fine-Tuning (SFT)

The simplest post-training method: fine-tune on high-quality instruction-response pairs.

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SFT}}} \left[\sum_t \log \pi_\theta(y_t \mid x, y_{<t}) \right]. \quad (7.1)$$

SFT teaches the model to follow instructions and adopt a desired interaction format. It does not, however, teach the model to refuse harmful requests or to express uncertainty—these require RL-based methods.

7.2 The RLHF Framework

Reinforcement Learning from Human Feedback (RLHF) consists of three phases:

1. **SFT**: initialise the policy π_{SFT} from the pre-trained base model.
2. **Reward modelling**: train $r_\phi(x, y)$ to predict human preferences:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{RM}}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]. \quad (7.2)$$

3. **Policy optimisation**: maximise reward with KL penalty:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot \mid x)} [r_\phi(x, y)] - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}}). \quad (7.3)$$

7.3 PPO (Proximal Policy Optimization)

PPO (Schulman et al., 2017) is the workhorse of RLHF:

Algorithm 7.1 (PPO-RLHF). For each iteration:

1. Sample trajectories $y \sim \pi_\theta(\cdot \mid x)$.
2. Compute advantages: $A_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} - V_\psi(x, y_{<t})$.

3. Clipped surrogate objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E} \left[\min \left(\frac{\pi_{\theta}(y_t | \dots)}{\pi_{\text{old}}(y_t | \dots)} A_t, \text{clip} \left(\frac{\pi_{\theta}}{\pi_{\text{old}}}, 1 - \epsilon, 1 + \epsilon \right) A_t \right) \right]. \quad (7.4)$$

4. Update value function V_{ψ} via MSE on returns.

5. Add KL penalty: $\mathcal{L} = \mathcal{L}_{\text{PPO}} - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{SFT}})$.

The critic network V_{ψ} estimates the value function (expected return from a state), reducing variance in the policy gradient estimate.

7.4 DPO (Direct Preference Optimization)

DPO (Rafailov et al., 2023) eliminates the reward model and critic, optimising the policy directly from preferences:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{SFT}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{SFT}}(y_l | x)} \right) \right]. \quad (7.5)$$

Theorem 7.1 (DPO Equivalence to RLHF). DPO is equivalent to RLHF under the Bradley-Terry preference model: the optimal policy π^* under the KL-constrained reward maximisation objective is identical to the policy obtained by DPO, up to the choice of β .

Sketch. The RLHF objective $\max_{\pi} \mathbb{E}[r(x, y)] - \beta D_{\text{KL}}(\pi \parallel \pi_{\text{SFT}})$ has closed-form solution:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{SFT}}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right). \quad (7.6)$$

Solving for r and substituting into the Bradley-Terry preference probability yields the DPO loss. $\square$

7.4.1 Likelihood Displacement Failure

Recent work (Razin et al., 2024) identifies a failure mode of DPO:

Theorem 7.2 (Likelihood Displacement). DPO can cause the policy to assign high probability to both preferred and dispreferred responses simultaneously, because the loss only penalises the *ratio* of probabilities, not their absolute values. This “likelihood displacement” degrades calibration.

7.5 GRPO (Group Relative Policy Optimization)

GRPO, used by DeepSeek-R1, eliminates the critic network entirely:

Algorithm 7.2 (GRPO). For each prompt x :

1. Sample G responses $\{y_1, \dots, y_G\} \sim \pi_{\theta}(\cdot | x)$.
2. Compute group-relative advantage:

$$A_i = \frac{r(x, y_i) - \mu_r}{\sigma_r}, \quad (7.7)$$

where $\mu_r = \frac{1}{G} \sum_j r(x, y_j)$, $\sigma_r = \text{std}(\{r(x, y_j)\})$.

3. Policy gradient update (no critic):

$$\nabla_{\theta} \mathcal{L}_{\text{GRPO}} = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) A_i - \beta \nabla_{\theta} D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{SFT}}) \right]. \quad (7.8)$$

Remark 7.1. GRPO replaces the learned value function with a Monte Carlo baseline computed from the group. This removes the critic network (halving memory during RL training) but introduces variance from finite group samples.

7.5.1 KL in the Loss vs. KL in the Reward

GRPO places the KL penalty inside the loss rather than inside the reward:

- **KL in reward:** $r'(x, y) = r(x, y) - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}})$. Used by PPO.
- **KL in loss:** $\mathcal{L} = \mathcal{L}_{\text{policy}} - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}})$. Used by GRPO, KTO.

When KL is in the loss, the KL penalty differentiates through the policy output, providing direct gradient signal to stay close to the reference model.

7.6 RLVR (Reinforcement Learning from Verifiable Rewards)

RLVR replaces the learned reward model with a rule-based verifier:

Definition 7.1 (Rule-Based Verifier). For problems with objectively correct answers (math, code), the verifier $v(x, y)$ returns 1 for correct and 0 for incorrect, based on:

- **Math:** comparing the final numerical answer.
- **Code:** executing the generated code against test cases.
- **Logic:** checking formal constraints.

RLVR applies any policy gradient method (PPO, GRPO) with $r(x, y) = v(x, y)$:

$$\max_{\pi_\theta} \mathbb{E}_{(x, y) \sim \pi_\theta} [v(x, y)] - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{SFT}}). \quad (7.9)$$

Remark 7.2. RLVR avoids reward hacking and reward model bias. DeepSeek-R1 uses RLVR for math and code, achieving significant reasoning improvements without human preference labels.

7.7 Comparison of RL Methods

Property	PPO	DPO	GRPO	RLVR
Reward model	Required	Not needed	Required	Rule-based
Critic network	Required	Not needed	Not needed	Optional
Online sampling	Yes	No (offline)	Yes	Yes
Memory (vs. SFT)	2×	1×	1.5×	1–2×
Gradient bias	Low	Likelihood displacement	Group-dependent	Verifier-dependent
KL placement	Reward	Implicit	Loss	Reward or loss
Best for	General alignment	Simple preference	Reasoning tasks	Math/code
Used by	GPT-4, Claude	Llama 3, Zephyr	DeepSeek-R1	DeepSeek-R1, Qwen

7.8 Other Alignment Methods

- **KTO (Kahneman-Tversky Optimisation):** uses only binary feedback (good/bad) rather than pairwise preferences. Loss derived from prospect theory.
- **SimPO (Simple Preference Optimization):** replaces the reference model with sequence-level likelihood, simplifying the DPO objective.

- **IPO (Identity Preference Optimization):** an alternative to DPO that avoids the log-ratio formulation and provides stronger guarantees against overfitting.
- **ORPO (Odds Ratio Preference Optimization):** combines SFT and preference optimisation into a single loss without a separate reference model.

7.8.1 K1 vs. K3 KL Estimators

The KL divergence in RLHF can be estimated with different sample sizes:

- **K1 estimator (per-token):** $\hat{D}_{\text{KL}}^{(1)} = \log \frac{\pi_{\theta}(y_t|\dots)}{\pi_{\text{SFT}}(y_t|\dots)}$. Low variance, biased per-token.
- **K3 estimator (per-sequence):** $\hat{D}_{\text{KL}}^{(3)} = \sum_t \log \frac{\pi_{\theta}(y_t|\dots)}{\pi_{\text{SFT}}(y_t|\dots)}$. Unbiased per-sequence, higher variance.

When KL is placed in the reward function, K1 is preferred because it provides a per-token KL penalty without introducing sequence-level noise. When KL is in the loss, K3 provides an unbiased estimate.

7.9 Exercises

1. Derive the closed-form optimal policy for the KL-constrained RLHF objective.
2. Show that DPO's loss function is equivalent to the RLHF objective under Bradley-Terry preferences.
3. Compare the memory requirements of PPO vs. DPO training for a 7B model.
4. Explain why GRPO does not need a critic network.
5. Simulate likelihood displacement in DPO with a synthetic preference dataset.
6. Design a rule-based verifier for a multi-step math word problem.

Part III

Beyond Transformers

Chapter 8

State Space Models and Alternative Architectures

“The quadratic cost of attention is not a law of nature; it is a design choice.”

Anonymous

The $O(n^2)$ scaling of attention motivates alternative architectures that achieve linear or sub-quadratic complexity while maintaining expressiveness.

8.1 From RNNs to Structured State Spaces

Recurrent neural networks (RNNs) process sequences in $O(n)$ time but suffer from vanishing gradients and limited memory. State space models (SSMs) reformulate recurrence as a continuous-time dynamical system:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t), \quad (8.1)$$

where $A \in \mathbb{R}^{d \times d}$ is the state transition matrix, $B \in \mathbb{R}^{d \times 1}$ the input projection, $C \in \mathbb{R}^{1 \times d}$ the output projection, and D a skip connection.

8.2 S4: Structured State Space Sequence Model

S4 (Gu et al., 2022) introduces a structured parametrisation of A :

Definition 8.1 (HiPPO Matrix). The HiPPO (High-Order Polynomial Projection Operators) matrix is:

$$A_{nk} = -\sqrt{(2n+1)(2k+1)} \cdot \begin{cases} 1 & \text{if } n > k, \\ n+1 & \text{if } n = k, \\ 0 & \text{if } n < k. \end{cases} \quad (8.2)$$

This matrix stores a compressed representation of the input history as coefficients of orthogonal polynomials.

S4 exploits the HiPPO matrix’s normal-plus-low-rank (NPLR) structure to diagonalise the recurrence, achieving $O(n)$ computation during training via convolution.

Theorem 8.1 (S4 Computational Complexity). S4 processes a sequence of length n in $O(n)$ time and $O(n)$ memory for training, and $O(1)$ per step for generation.

8.3 H3: Leveraging SSMs for Language

H3 (Hungry Hungry Hippo; Fu et al., 2023) extends S4 by adding multiplicative gates inspired by attention:

- Two SSMs in series with a gating mechanism.
- The first SSM produces a query-like signal; the second produces a key-like signal.
- Element-wise multiplication of the two SSM outputs creates a content-based interaction.

H3 achieves competitive results with transformers on language modelling while maintaining linear complexity.

8.4 Mamba: S6 Selective SSM

Mamba (Gu & Dao, 2023) introduces the S6 layer, which makes the SSM parameters input-dependent:

$$B_t = \text{Linear}_B(x_t), \quad C_t = \text{Linear}_C(x_t), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t)). \quad (8.3)$$

Definition 8.2 (Selectivity). A model is *selective* if its state-space dynamics depend on the input content, not just position. This enables content-aware recall (e.g., “ignore filler tokens, remember the key entity”), which was impossible in LTI SSMs.

Selectivity breaks the convolution equivalence: Mamba cannot be computed via convolution because B_t and C_t vary per token. Instead, Mamba uses a hardware-aware parallel scan:

Algorithm 8.1 (Mamba Parallel Scan). 1. Compute $\bar{A}_t = \exp(\Delta_t \cdot A)$, $\bar{B}_t = \Delta_t \cdot B_t$.
 2. Scan recurrence: $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$.
 3. Parallelise via associative scan: the recurrence $(h_t, \bar{A}_t, \bar{B}_t x_t)$ forms a semigroup under composition.
 4. Compute $y_t = C_t h_t$.

The parallel scan runs in $O(\log n)$ steps on $O(n)$ processors, achieving practical speedups over attention for long sequences.

8.5 Mamba-2: SSD Reformulation

Mamba-2 (Dao & Gu, 2024) reformulates the SSM as a structured state-space duality (SSD):

Theorem 8.2 (SSD = Attention + SSM). The Mamba-2 layer computes:

$$y = (L \circ S) \cdot v, \quad (8.4)$$

where L is a causal mask (lower-triangular), $S = QK^\top$ is a structured semi-separable matrix, and $\circ$ denotes element-wise multiplication.

This reveals that Mamba-2’s SSM is a *linear attention* variant with a structured kernel. Key innovations:

- **Multi-head SSM:** multiple independent SSM heads, matching attention’s multi-head structure.
- **Head dimension:** $d_{\text{head}} = 64$ or 128 , matching typical attention configurations.
- **State dimension:** $N = 64$ – 256 per head, larger than Mamba-1’s $N = 16$.

The SSD formulation enables efficient training via matrix multiplication ($O(n^2)$ on GPU, matching attention), while retaining $O(1)$ per-step generation.

8.6 Mamba-3: Complex States and MIMO

Mamba-3 (2025) introduces three major advances:

- **Exponential-trapezoidal discretisation:** higher-order integration of the continuous SSM reduces approximation error.
- **Complex-valued states:** $A \in \mathbb{C}^{d \times d}$ with complex eigenvalues enables oscillatory dynamics and richer temporal patterns.
- **MIMO (multi-input multi-output):** each SSM processes multiple input and output channels simultaneously, increasing expressiveness per parameter.

Theorem 8.3 (Mamba-3 Expressiveness). With complex-valued states, Mamba-3 can represent any linear dynamical system of dimension $2d$, whereas real-valued Mamba-2 requires $2d$ real dimensions.

8.7 Hybrid Architectures

Several models combine SSMs with attention for the best of both worlds:

- **Jamba (AI21, 2024):** interleaves Mamba layers with attention layers and MoE FFNs. Attention layers handle long-range recall; Mamba layers handle efficient local processing.
- **TransMamba:** shares query/key/value projections between the attention and SSM paths, using a cross-block residual (CBx) connection. Params: 91% shared, 9% unique.
- **RWKV:** linear attention reformulated as a recurrent neural network with a time-dependent decay factor. $O(n)$ training via WKV computation.
- **RetNet (Sun et al., 2023):** retention mechanism with three computation modes—parallel (training), recurrent (generation), chunkwise (efficient decoding).

8.8 MoE-SSM Hybrids

- **Swimba (2025):** replaces FFN experts in MoE with SSM experts. The router selects which SSM to apply per token. Single-pass recurrence (no KV cache) enables extreme throughput.
- **Nemotron-H (NVIDIA, 2025):** hybrid MoE with both attention and SSM experts, using hardware-aware scheduling to balance workload across experts.

8.9 Comparison with Transformers

Property		Transformer	Mamba-2	Jamba
Training	com-	$O(n^2d)$	$O(ndN)$	Mixed
plexity				
Generation	com-	$O(nd)$ per step	$O(1)$ per step	$O(1)$ per step
plexity				
Memory	(long	$O(n^2)$	$O(nd)$	$O(nd)$
context)				
Content-based re-		Yes (attention)	Yes (selective)	Yes
call				
Long-range	de-	Unlimited	State-limited (N)	Hybrid
pendencies				

Throughput (long seq.)	Degrades	Stable	Near-stable
------------------------	----------	--------	-------------

8.10 Exercises

1. Show that S4 can be computed via convolution when A, B, C are time-invariant.
2. Derive the associative scan for the Mamba recurrence $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$.
3. Explain why selectivity breaks the convolution equivalence in Mamba.
4. Prove that the SSD formulation in Mamba-2 is equivalent to linear attention with a structured kernel.
5. Compare the generation-time memory of a transformer (KV cache) vs. Mamba (state vector).
6. Design a hybrid architecture that interleaves 1 attention layer every 4 Mamba layers. Justify the ratio.

Part IV

Inference

Chapter 9

Autoregressive Inference

9.1 Decoding Strategies

Definition 9.1 (Greedy Decoding). At each step t , select

$$x_t^* = \arg \max_{x \in \mathcal{V}} p_\theta(x \mid x_{<t}). \quad (9.1)$$

Greedy decoding is deterministic but may miss high-probability continuations due to locally optimal choices.

Definition 9.2 (Beam Search). Maintain a set $\mathcal{B}_t$ of k partial hypotheses. At each step, expand each hypothesis with the top- k tokens and retain the k highest-scoring sequences:

$$\mathcal{B}_{t+1} = \text{top-}k \left(\bigcup_{y \in \mathcal{B}_t} \bigcup_{x \in \mathcal{V}} \{(y, x)\} \text{ with score } \log p_\theta(y, x) \right). \quad (9.2)$$

Definition 9.3 (Top- k Sampling). Sample from the k highest-probability tokens:

$$p'(x \mid x_{<t}) = \begin{cases} p(x \mid x_{<t}) / \sum_{i=1}^k p(i) & \text{if } x \in \mathcal{V}^{(k)}, \\ 0 & \text{otherwise,} \end{cases} \quad (9.3)$$

where $\mathcal{V}^{(k)}$ is the set of k tokens with highest probability.

Definition 9.4 (Top- p (Nucleus) Sampling). Sample from the smallest set $\mathcal{V}^{(p)}$ such that

$$\sum_{x \in \mathcal{V}^{(p)}} p(x \mid x_{<t}) \geq p. \quad (9.4)$$

Nucleus sampling adaptively truncates the tail of the distribution.

9.2 Temperature Sampling

Temperature τ rescales the logits before the softmax:

$$p_\tau(x \mid x_{<t}) = \frac{\exp(\ell_t/\tau)}{\sum_{x'} \exp(\ell_{x'}/\tau)}. \quad (9.5)$$

As $\tau \rightarrow 0$, sampling approaches greedy decoding. As $\tau \rightarrow \infty$, sampling approaches uniform.

Theorem 9.1 (Temperature and Entropy). The entropy of the temperature-scaled distribution satisfies:

$$H(p_\tau) = \log |\mathcal{V}| - \frac{1}{\tau} \sum_x p(x) \log p(x) + O(\tau^{-2}) \quad \text{as } \tau \rightarrow \infty. \quad (9.6)$$

Lower temperatures concentrate probability mass, reducing diversity.

9.3 The KV Cache

During autoregressive generation, the key and value tensors for all previous tokens are cached to avoid recomputation:

$$K_{\leq t} = [K_{\leq t-1}; K_t] \in \mathbb{R}^{t \times d_k}, \quad (9.7)$$

$$V_{\leq t} = [V_{\leq t-1}; V_t] \in \mathbb{R}^{t \times d_v}. \quad (9.8)$$

The KV cache has memory cost:

$$M_{KV} = 2 \cdot L \cdot n_{kv} \cdot t \cdot d_h \cdot p \quad \text{bytes}. \quad (9.9)$$

For a 70B model with $t = 4096$, $L = 80$, $n_{kv} = 8$, $d_h = 128$, $p = 2$ (FP16): $M \approx 1.34$ GB per sequence.

9.4 Parallel Decoding

Several techniques generate multiple tokens per step:

- **Medusa (Cai et al., 2024)**: add k parallel heads that predict the next k tokens simultaneously. Each head predicts one future position. Use tree attention to process all hypotheses.
- **Lookahead Decoding (Fu et al., 2024)**: use Jacobi iteration to generate k tokens per step. Each step, the model predicts all k positions based on current guesses, then updates.
- **Eagle (Li et al., 2024)**: a lightweight draft model predicts future tokens using both the current hidden state and previously predicted tokens.

These methods achieve 2–4× speedup in wall-clock time at the cost of increased FLOPs.

9.5 Exercises

1. Compare greedy decoding and beam search on a sequence where the locally optimal choice leads to a dead end.
2. Compute the KV cache memory for Llama 3 70B at $t = 8192$.
3. Show that temperature $\tau = 1$ recovers the original distribution.
4. Design a Medusa head architecture for 4-token lookahead.
5. Explain why nucleus sampling is preferred over top- k for diverse generation.

Chapter 10

Inference Optimisation

10.1 Quantisation

Definition 10.1 (Uniform Quantisation). Map a weight $w \in [\alpha, \beta]$ to b -bits:

$$\hat{w} = \Delta \cdot \text{clamp} \left(\left\lfloor \frac{w - \alpha}{\Delta} \right\rfloor, 0, 2^b - 1 \right) + \alpha, \quad (10.1)$$

where $\Delta = (\beta - \alpha)/(2^b - 1)$.

10.1.1 GPTQ

Optimal Brain Quantiser (Frantar et al., 2023): iteratively quantise weights while compensating the remainder using the inverse Hessian:

$$\delta w_q = -\frac{w_q}{\text{diag}(H_{qq}^{-1})} \cdot H_{:,q}^{-1}, \quad (10.2)$$

where $H = 2XX^\top$ is the Hessian of the MSE loss.

10.1.2 AWQ

Activation-aware weight quantisation (Lin et al., 2023): scale salient weights (those corresponding to large activations) before quantisation to preserve outlier channels:

$$\tilde{w} = w \cdot s, \quad \text{Quantise}(\tilde{w}), \quad \hat{w} = \text{Dequantise}(\hat{\tilde{w}}) \cdot s^{-1}. \quad (10.3)$$

The per-channel scale s is chosen to minimise MSE under a small calibration set.

10.1.3 GGUF and NF4

For CPU inference, llama.cpp uses the GGUF format with various quantisation levels:

- **GGUF Q4_K_M**: 4-bit quantisation with importance-based mixed precision (some weights get 5 or 6 bits).
- **NF4 (QLoRA)**: normalised float 4-bit, where quantisation levels match a standard normal distribution: $\text{NF4} = \{-1.0, -0.85, -0.7, \dots, 0.85, 1.0\}$.

10.1.4 FP8 Inference (H100)

The H100 GPU supports FP8 matrix multiplication natively:

- **E4M3** (4 exponent bits, 3 mantissa): range ± 448 , fine-grained for weights.
- **E5M2** (5 exponent bits, 2 mantissa): wider range ± 57344 , coarser granularity, suitable for activations.
- FP8 inference achieves $2\times$ throughput vs. FP16 with minimal quality loss ($< 0.5\%$ perplexity increase).

10.2 Speculative Decoding

Speculative decoding (Leviathan et al., 2022; Chen et al., 2023) accelerates generation by using a small draft model M_q to propose K tokens, which are then verified by the target model M_p in parallel.

Algorithm 10.1 (Speculative Decoding). 1. Draft model M_q generates K tokens autoregressively: $x_{t+1}, \dots, x_{t+K} \sim M_q(\cdot \mid x_{\leq t})$.
 2. Target model M_p computes logits for all $K + 1$ prefixes in parallel.
 3. For each position $i = 1, \dots, K$, accept x_{t+i} with probability $\min(1, p(x_{t+i})/q(x_{t+i}))$.
 4. Reject at the first rejection; resample from the residual distribution $\propto \max(0, p - q)$.

The expected speedup is:

$$\mathbb{E}[\text{speedup}] = \frac{\sum_{k=1}^K k \cdot P(\text{accept } k)}{\text{latency overhead}}. \quad (10.4)$$

Theorem 10.1 (Acceptance Rate). The expected number of accepted tokens is:

$$\mathbb{E}[K_{\text{acc}}] = \sum_{k=1}^K P(\text{accept } k) = \sum_{k=1}^K \prod_{i=1}^k \min\left(1, \frac{p(x_{t+i})}{q(x_{t+i})}\right). \quad (10.5)$$

When $M_q \approx M_p$, $\mathbb{E}[K_{\text{acc}}] \approx K$.

10.3 Flash Attention

Flash Attention (Dao et al., 2022) computes exact attention without materialising the $N \times N$ matrix, using tiling and recomputation:

1. Tile Q , K , V into blocks that fit in SRAM.
2. For each block, compute the partial softmax and update the output incrementally using online softmax.
3. Recompute the attention matrix during the backward pass (instead of storing it) to trade FLOPs for memory bandwidth.

This reduces memory from $O(N^2)$ to $O(N)$ and achieves 2–4× speedup on GPUs.

Theorem 10.2 (Flash Attention IO Complexity). Flash Attention achieves $O(N^2 d^2 / M)$ HBM accesses, where M is the SRAM size, compared to $O(N^2 d + N d^2)$ for standard attention. For $M = 108\text{KB}$ (A100), this is a 5–10× reduction in HBM traffic.

10.4 Continuous Batching and PagedAttention

- **Continuous batching:** instead of waiting for all sequences in a batch to complete, new sequences are admitted as soon as existing ones finish. This maximises GPU utilisation.
- **PagedAttention (vLLM):** KV cache blocks are stored in non-contiguous physical memory pages, managed like virtual memory. This eliminates internal fragmentation and enables copy-on-write for parallel sampling.

10.5 Exercises

1. Show that 4-bit quantisation compresses model weights by a factor of 8× relative to FP16.
2. Derive the expected speedup of speculative decoding when the acceptance rate is α .
3. Explain why Flash Attention reduces memory from $O(N^2)$ to $O(N)$.

4. Compare continuous batching with static batching in terms of throughput and latency.
5. Compute the FP8 memory savings for a 70B model vs. FP16.
6. Show that NF4 quantisation preserves the expected value of standard-normal-distributed weights.

Chapter 11

Long-Context Techniques

“Context length is not a ceiling, it is a floor.”

Anonymous

Standard RoPE has limited ability to generalise beyond the training sequence length. This chapter covers techniques to extend the context window of pre-trained LLMs.

11.1 Position Interpolation (PI)

PI (Chen et al., 2023) rescales position indices to fit within the trained range:

$$\theta'_j = \theta_j \cdot \frac{L_{\text{train}}}{L_{\text{target}}}, \quad (11.1)$$

where L_{train} is the training context length and L_{target} is the desired length.

Definition 11.1 (Position Interpolation). For a target length $L > L_{\text{train}}$, the position m is mapped to $m' = m \cdot (L_{\text{train}}/L)$. All RoPE frequencies remain the same but positions are compressed.

PI requires fine-tuning on the extended length but causes resolution loss for nearby tokens.

11.2 NTK-Aware Scaling

NTK-aware scaling (bloc97, 2023) scales high-frequency components less and low-frequency components more, motivated by the Neural Tangent Kernel:

$$\theta'_j = \theta_j \cdot \alpha^{2j/d}, \quad (11.2)$$

where $\alpha = (L_{\text{target}}/L_{\text{train}})^{d/(d-2)}$.

Remark 11.1. NTK-aware scaling preserves high-frequency information (local token interactions) while extending the range of low-frequency components (long-range dependencies). It requires minimal or no fine-tuning.

11.3 YaRN (Yet another RoPE extensioN)

YaRN (Peng et al., 2023) combines PI and NTK-aware scaling with a ramp function:

$$\theta'_j = \begin{cases} \theta_j \cdot s & \text{if } j < r \text{ (low freq.)}, \\ \theta_j & \text{if } j > r + w \text{ (high freq.)}, \\ \text{interpolated} & \text{otherwise,} \end{cases} \quad (11.3)$$

$$\text{where } s = \frac{L_{\text{target}}}{L_{\text{train}}}. \quad (11.4)$$

Key parameters:

- **Ramp length w :** width of the transition region (typically $w = 8$).
- **Ramp start r :** dimension below which full interpolation is applied.
- **Temperature factor t :** rescales attention logits: $t = \sqrt{\log(L_{\text{target}}/L_{\text{train}})}$.

Theorem 11.1 (YaRN Frequency Groups). YaRN partitions dimensions into three groups:

1. **High-frequency:** $j > r + w$, keep original θ_j (no interpolation).
2. **Low-frequency:** $j < r$, apply full position interpolation.
3. **Ramp region:** $r \leq j \leq r + w$, smoothly transition between the two.

This preserves local information while extending range.

11.4 Dynamic NTK

Dynamic NTK applies NTK-aware scaling at inference time without fine-tuning:

$$\alpha = \max\left(1, \frac{L_{\text{current}}}{L_{\text{train}}}\right), \quad (11.5)$$

where L_{current} is the actual sequence length at each inference step. The scaling factor changes dynamically as generation progresses.

11.5 LongRoPE

LongRoPE (2024) uses evolutionary search to find per-dimension position interpolation factors:

$$\theta'_j = \frac{\theta_j}{s_j}, \quad (11.6)$$

where s_j is the per-dimension scale, found via:

$$\min_{s_1, \dots, s_{d/2}} \text{PPL}(\text{model with scales } s_j), \quad \text{s.t. } s_j \text{ monotonically non-decreasing in } j. \quad (11.7)$$

Definition 11.2 (Monotonic Constraint). The scale factors s_j must be monotonically non-decreasing: $s_1 \leq s_2 \leq \dots \leq s_{d/2}$. This ensures that lower-frequency dimensions (larger j) are interpolated more, consistent with the NTK intuition.

LongRoPE extends context to 256K with minimal perplexity degradation and no fine-tuning beyond a small calibration set.

11.6 LongRoPE2

LongRoPE2 (2025) refines the evolutionary search with a needle-driven objective:

$$\min_s \text{PPL}_{\text{needle}}(s) = \text{PPL}(\text{needle-in-haystack with scales } s). \quad (11.8)$$

Instead of perplexity on long documents, LongRoPE2 optimises for the needle-in-haystack retrieval success rate, directly targeting the use case that long-context models must support.

11.7 AlphaRoPE

AlphaRoPE uses power-function scaling as a compromise between PI and NTK:

$$\theta'_j = \theta_j \cdot \left(\frac{L_{\text{target}}}{L_{\text{train}}} \right)^{\alpha_j}, \quad \alpha_j = 1 - \frac{2j}{d}. \quad (11.9)$$

High-frequency dimensions ($j \approx 0$) get $\alpha_j \approx 1$ (full PI scaling), while low-frequency dimensions ($j \approx d/2$) get $\alpha_j \approx 0$ (no scaling). This smooth interpolation avoids the hard ramp boundary of YaRN.

11.8 CoPE (Contextual Position Encoding)

CoPE (2025) applies soft clipping to low-frequency components:

$$\theta'_j = \theta_j \cdot \text{clip} \left(\frac{L_{\text{target}}}{L_{\text{train}}}, 1, \tau_j \right), \quad (11.10)$$

where τ_j is a per-dimension clipping threshold learned or defined as $\tau_j = 1 + (c - 1) \cdot \frac{2j}{d}$.

CoPE achieves 2× perplexity improvement over PI on long-context benchmarks without fine-tuning.

11.9 RoPE Out-of-Distribution Theory

Recent theoretical work unifies these techniques:

Theorem 11.2 (RoPE OOD Decomposition). The OOD error from length extrapolation decomposes into:

$$\mathcal{E}_{\text{ext}} = \mathcal{E}_{\text{freq}} + \mathcal{E}_{\text{pos}} + \mathcal{E}_{\text{atten}}, \quad (11.11)$$

where:

- $\mathcal{E}_{\text{freq}}$: frequency mismatch between trained and novel positions.
- $\mathcal{E}_{\text{pos}}$: absolute position encoding drift beyond trained range.
- $\mathcal{E}_{\text{atten}}$: attention entropy collapse at unseen positions.

NTK-aware scaling reduces $\mathcal{E}_{\text{freq}}$, YaRN’s temperature factor reduces $\mathcal{E}_{\text{atten}}$, and LongRoPE’s per-dimension scales reduce $\mathcal{E}_{\text{pos}}$.

11.10 Comparison

Method	Max length	Fine-tune needed?	Key idea	PPL @ 128K
None	L_{train}	No	–	Degrades
PI	$2 \times L_{\text{train}}$	Yes	Linear rescaling	+1.5
NTK-aware	$4 \times L_{\text{train}}$	No	Frequency-dependent	+0.8
YaRN	$16 \times L_{\text{train}}$	No	Ramp + temperature	+0.3
Dynamic NTK	$8 \times L_{\text{train}}$	No	Evolving scale	+0.6
LongRoPE	$32 \times L_{\text{train}}$	Minimal	Per-dimension search	+0.1
AlphaRoPE	$16 \times L_{\text{train}}$	No	Power-function	+0.4

CoPE	$16 \times L_{\text{train}}$	No	Soft clipping	+0.2
------	------------------------------	----	---------------	------

11.11 Exercises

1. Derive the position interpolation formula for RoPE.
2. Explain why NTK-aware scaling preserves high-frequency components.
3. Implement the YaRN ramp function in pseudocode.
4. Show that LongRoPE's monotonic constraint is consistent with NTK intuition.
5. Compare the fine-tuning requirements of PI vs. YaRN vs. LongRoPE.
6. Propose a new RoPE extension method combining AlphaRoPE and CoPE.

Part V

Engineering

Chapter 12

GPU Architecture for LLMs

12.1 GPU Memory Hierarchy

Modern GPUs (NVIDIA Hopper, Ada Lovelace) have a hierarchical memory system:

- **Global memory (HBM3/HBM2e):** 40–144 GB, bandwidth 2–3.35 TB/s. High latency (~ 600 cycles).
- **L2 cache:** 40–60 MB on-chip. Shared across SMs.
- **Shared memory / SRAM:** 128–256 KB per SM. Low latency (~ 20 cycles), explicitly managed.
- **Registers:** 65536 per SM. Zero-latency access.

12.2 Compute vs. Memory Bound Analysis

An operation is:

- **Compute-bound** if arithmetic intensity $> \frac{\text{peak FLOPS}}{\text{memory bandwidth}}$ (e.g., FFN layers).
- **Memory-bound** otherwise (e.g., attention with KV cache, embedding lookup).

For an A100 (312 TFLOPS FP16, 2 TB/s HBM bandwidth), the threshold is $I_{\text{crit}} = 312/2 = 156$ FLOPs/byte. FFN layers: $I \approx 200$ FLOPs/byte (compute-bound). Attention: $I \approx 20$ FLOPs/byte (memory-bound).

12.3 Tensor Cores and Mixed Precision

Tensor Cores perform 4×4 matrix multiply-accumulate in one cycle:

$$D = A \cdot B + C, \quad (12.1)$$

where A and B are FP16/BF16/INT8 and D and C are FP16/INT32.

Mixed-precision training uses FP16/BF16 for forward/backward with FP32 master weights for numerical stability.

12.3.1 Hopper-Specific Features

NVIDIA H100 introduces:

- **FP8 Tensor Cores:** $2\times$ throughput vs. FP16, with E4M3/E5M2 formats.
- **Transformer Engine:** automatic FP8/FP16 selection per layer based on activation statistics.
- **DPX instructions:** accelerate dynamic programming (Smith-Waterman, Floyd-Warshall) used in some sequence alignment tasks.
- **Asynchronous transactions:** overlapping data movement with computation via thread block clusters.

12.4 Parallelism Strategies

- **Tensor parallelism (TP):** each layer’s parameters are sharded across GPUs. Requires all-reduce per sub-layer. Optimal when intra-node NVLink bandwidth is high.
- **Pipeline parallelism (PP):** layers are partitioned into stages. Bubble overhead is $\frac{(P-1) \cdot (M-1)}{P \cdot M}$ where P is the number of pipeline stages and M is the number of micro-batches.
- **Sequence parallelism:** the sequence dimension is split across GPUs. Each GPU processes a segment of the sequence.
- **Expert parallelism (MoE):** each expert is placed on a different GPU; tokens are routed via all-to-all communication.

12.4.1 Interconnects

- **NVLink (H100):** 900 GB/s bidirectional per GPU, direct GPU-to-GPU.
- **InfiniBand (HDR200):** 400 Gb/s per link, used for inter-node communication.
- **PCIe Gen5:** 128 GB/s per direction, used for host-to-GPU transfer.

12.5 Kernel Fusion

Launching many small GPU kernels incurs overhead. Kernel fusion combines operations:

- **Fused attention:** combine $QK^\top$, softmax, dropout, SV into one kernel.
- **Fused layer norm + residual + quantisation:** one kernel for all three.
- **Flash Attention** (see §10.3) is the canonical fused attention kernel.

12.6 Exercises

1. Compute the arithmetic intensity of a single transformer FFN layer.
2. Explain why memory-bound operations benefit more from kernel fusion.
3. Compare the communication costs of TP and PP for a 70B model on 8 GPUs.
4. Calculate the HBM bandwidth utilisation for an H100 running FP8 attention at batch size 1, sequence length 8192.
5. Show that the roofline model predicts compute-bound behaviour for FFN layers but memory-bound for attention.

Chapter 13

Software Engineering for LLMs

13.1 Prompt Engineering Patterns

Definition 13.1 (Chain-of-Thought (CoT)). Elicit intermediate reasoning before the final answer:

```
Q: {question}
A: Let's think step by step.
```

Definition 13.2 (Few-Shot Prompting). Provide k exemplars of the task in the prompt:

```
Input: x_1 -> Output: y_1
Input: x_2 -> Output: y_2
...
Input: x_k -> Output: y_k
Input: x -> Output:
```

Definition 13.3 (Structured Output Parsing). Request machine-parseable output formats:

```
Output in JSON:
{
  "answer": "...",
  "reasoning": "...",
  "confidence": 0.95
}
```

13.2 Retrieval-Augmented Generation (RAG)

RAG augments LLM input with retrieved documents:

$$p(y \mid x) = \sum_{d \in \text{Retrieve}(x)} p_{\text{LLM}}(y \mid x, d) p_{\text{retriever}}(d \mid x). \quad (13.1)$$

The canonical RAG pipeline:

1. **Indexing:** chunk documents into passages; embed with a dense retriever (e.g., BERT, E5).
2. **Retrieval:** given query q , find top- k passages by cosine similarity: top- $k \cos(E(q), E(D))$.
3. **Generation:** concatenate query and retrieved passages as LLM input.

13.2.1 Advanced RAG Patterns

- **Hierarchical retrieval:** first retrieve documents, then passages within documents.
- **Query rewriting:** use the LLM to rewrite the query before retrieval.
- **HyDE (Hypothetical Document Embedding):** generate a hypothetical answer, then retrieve based on that answer's embedding.
- **Iterative retrieval:** after each generation step, retrieve additional context.
- **Self-RAG:** the LLM decides when to retrieve, what to retrieve, and whether to use retrieved information.

13.3 Agent Architectures

LLM agents extend the model with tools and structured reasoning:

```
def agent_loop(query):
    messages = [system_prompt, user_message(query)]
    while not done:
        response = llm(messages)
        action = parse_action(response)
        if action.type == "call_tool":
            result = execute_tool(action.name, action.args)
            messages.append({"role": "tool", "content": result})
        elif action.type == "final_answer":
            return action.content
```

13.4 Testing and Evaluation

- **Unit testing:** test individual prompt templates, parsing logic, and tool integrations.
- **Regression testing:** maintain a set of canonical inputs and expected outputs; measure pass rate across model versions.
- **Evaluation suites:** MMLU (knowledge), HumanEval (code), MT-Bench (conversation), HELM (holistic).
- **Hallucination detection:** measure token-level perplexity, semantic entropy, or self-consistency across multiple samples.

13.5 Production Deployment Patterns

- **Serve with an inference engine:** vLLM, TGI, TensorRT-LLM.
- **Load balancing:** round-robin or least-connections across model replicas.
- **Caching:** KV cache reuse for common prefixes; semantic caching for identical queries.
- **Monitoring:** latency, throughput, TTFT (time to first token), ITL (inter-token latency).
- **Guardrailing:** input/output content filtering, PII detection, topic blocking.

13.6 Exercises

1. Implement a RAG pipeline pseudocode with chunking, embedding, retrieval, and generation.
2. Design an agent that can use a calculator, a web search tool, and a database query tool.
3. Propose a testing strategy for an LLM-based code generation system.
4. Compare the latency impact of adding a retrieval step to an LLM call.

Part VI

Practice

Chapter 14

Libraries and Frameworks

14.1 HuggingFace Transformers

The de facto standard library for LLMs:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16,
    device_map="auto",
    attn_implementation="flash_attention_2",
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")
```

Key features: model hub, pipeline API, trainer, quantisation (bitsandbytes), PEFT (LoRA).

14.2 vLLM

High-throughput serving with PagedAttention:

```
from vllm import LLM, SamplingParams

llm = LLM("meta-llama/Llama-3.1-8B-Instruct")
params = SamplingParams(temperature=0.7, top_p=0.9, max_tokens=512)
outputs = llm.generate(prompts, params)
```

Key features: continuous batching, PagedAttention, prefix caching, tensor parallelism, AWQ/GPTQ quantisation, OpenAI-compatible API server.

14.3 TensorRT-LLM

NVIDIA's optimised inference framework:

```
# Build engine
trtllm-build --model_dir llama-8b \
  --dtype bfloat16 \
  --max_batch_size 64 \
  --max_input_len 4096 \
  --max_output_len 1024

# Run server
python scripts/launch_triton_server.py \
  --model_repository=/models/llama-8b \
```

```
--world_size=4
```

Key features: FP8 quantisation, in-flight batching, multi-GPU tensor parallelism, CUDA graph optimisation, speculative decoding.

14.4 llama.cpp

CPU-first inference with GGUF quantised models:

```
./llama-cli -m llama-8b.Q4_K_M.gguf \
-p "Once upon a time" \
-n 256 -t 8
```

Key features: GGUF quantisation (2–8 bit), CPU/GPU hybrid, Metal/CUDA/Vulkan back-ends, no Python dependency.

14.5 LangChain

Framework for LLM application development:

```
from langchain.chains import RetrievalQA
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings

retriever = FAISS.from_texts(docs, HuggingFaceEmbeddings()).as_retriever()
qa = RetrievalQA.from_llm(llm, retriever=retriever)
result = qa.invoke("What is the capital of France?")
```

Key features: modular chains, agent frameworks, tool integration, memory, and streaming.

14.6 LlamaIndex

Data framework for RAG applications:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("data/").load_data()
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()
response = query_engine.query("What does the document say about X?")
```

Key features: advanced indexing strategies (tree, keyword, hybrid), query routing, document agents, structured data extraction.

14.7 Comparison Table

Framework	Use Case	Quantisation	Batching	GPU req.
HF Transformers	Research, proto- typing	GPTQ, AWQ, bitsandbytes	Static	≥ 1 GPU
vLLM	Production serv- ing	GPTQ, AWQ, FP8	Continuous	≥ 1 GPU
TensorRT-LLM	Max throughput	FP8, INT4, INT8	In-flight	≥ 1 GPU

llama.cpp	Local/edge inference	GGUF 2–8 bit	Static	CPU/any GPU
LangChain	Application dev	N/A (uses back-end)	N/A	N/A
LlamaIndex	RAG systems	N/A (uses back-end)	N/A	N/A

14.8 Exercises

1. Deploy a Llama 3.1 8B model using vLLM with tensor parallelism on 2 GPUs.
2. Quantise a model to 4-bit using AWQ and compare the perplexity vs. FP16.
3. Build a RAG pipeline with LlamaIndex that retrieves from a PDF collection.
4. Compare the throughput of vLLM vs. llama.cpp for a 7B model on the same hardware.

Chapter 15

Evaluation and Advanced Topics

15.1 Benchmark Suites

Benchmark	What it measures	Format
MMLU	Multi-task knowledge (57 subjects)	4-way MCQ
HumanEval	Code generation correctness	Function completion
MT-Bench	Multi-turn conversation quality	LLM-as-judge scoring
GSM8K	Grade-school math reasoning	Chain-of-thought
HELM	Holistic evaluation (42 scenarios)	Multi-metric
BigBench	Diverse reasoning tasks	Varied
HellaSwag	Commonsense reasoning	Sentence completion

15.2 Hallucination and Calibration

An LLM is *calibrated* if its confidence matches its accuracy:

$$\mathbb{P}(\text{correct} \mid p_{\max} = c) = c \quad \forall c \in [0, 1]. \quad (15.1)$$

Calibration is measured with expected calibration error (ECE):

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} \left| \text{acc}(B_m) - \overline{\text{conf}}(B_m) \right|, \quad (15.2)$$

where bins B_m partition $[0, 1]$.

15.2.1 Hallucination Taxonomies

- **Factual hallucination:** model asserts false information (e.g., “The Eiffel Tower is in London”).
- **Contextual hallucination:** model contradicts provided context (e.g., ignoring retrieved documents).
- **Reasoning hallucination:** model makes logical errors in multi-step reasoning.
- **Instruction hallucination:** model performs a different task than instructed.

Hallucination detection methods:

- **Semantic entropy:** sample multiple answers, cluster by meaning, measure dispersion.

- **Self-consistency:** chain-of-thought samples reach the same answer.
- **P(True):** prompt the model to evaluate its own answer's correctness.
- **Context grounding:** verify that each claim is supported by the retrieved context.

15.3 Safety Evaluation

- **Red-teaming:** adversarial testing to find failure modes.
- **Refusal rate:** fraction of harmful prompts correctly refused.
- **Sycophancy:** tendency to agree with user even when wrong.
- **Benchmark contamination:** test set overlap with training data inflates scores.

15.4 Outlook: The Future of LLM Engineering

The field is evolving rapidly. Key directions:

- **MoE architecture:** Mixture-of-Experts enables larger models with constant inference cost.
- **Long context:** 1M+ token context windows are becoming practical.
- **Multimodality:** native image, video, and audio understanding.
- **Agentic workflows:** LLMs as autonomous agents with tool use and planning.
- **Hardware co-design:** custom accelerators (Groq, Cerebras, Etched) for transformer inference.
- **Test-time compute scaling:** spending more compute at inference (CoT, search, self-consistency) improves reasoning quality.

15.5 Final Project

1. Select an open-source LLM and deploy it with vLLM.
2. Implement a structured extraction pipeline that parses JSON outputs.
3. Build an agent with tool calling (calculator, search, database).
4. Measure latency, throughput, and TTFT under varying batch sizes.
5. Quantise the model to 4-bit and measure the quality degradation.
6. Write a report summarising the trade-offs observed.

Bibliography

- [1] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, 1948.
- [2] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [3] J. Kaplan et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [4] J. Hoffmann et al., “Training Compute-Optimal Large Language Models,” *NeurIPS*, 2022.
- [5] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022.
- [6] Y. Leviathan et al., “Fast Inference from Transformers via Speculative Decoding,” *ICML*, 2022.
- [7] C. Chen et al., “Accelerating Large Language Model Decoding with Speculative Sampling,” *arXiv:2302.01318*, 2023.
- [8] E. Frantar et al., “GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers,” *ICLR*, 2023.
- [9] J. Lin et al., “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” *MLSys*, 2023.
- [10] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *SOSP*, 2023.
- [11] L. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” *NeurIPS*, 2022.
- [12] J. W. Rae et al., “Scaling Language Models: Methods, Analysis & Insights from Training Gopher,” *arXiv:2112.11446*, 2022.
- [13] H. Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [14] A. Dubey et al., “The Llama 3 Herd of Models,” *arXiv:2407.21783*, 2024.
- [15] A. Q. Jiang et al., “Mistral 7B,” *arXiv:2310.06825*, 2023.
- [16] N. Tishby et al., “The Information Bottleneck Method,” *Allerton Conference*, 1999.
- [17] J. Perez et al., “True Few-Shot Learning with Language Models,” *NeurIPS*, 2021.
- [18] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *NeurIPS*, 2022.

- [19] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [20] R. Rafailov et al., “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” *NeurIPS*, 2023.
- [21] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [22] A. Gu & T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [23] T. Dao & A. Gu, “Transformers are SSMS: Generalized Models and Efficient Algorithms,” *arXiv:2405.21060*, 2024.
- [24] A. Gu et al., “Efficiently Modeling Long Sequences with Structured State Spaces,” *ICLR*, 2022.
- [25] R. Sennrich et al., “Neural Machine Translation of Rare Words with Subword Units,” *ACL*, 2016.
- [26] T. Kudo & J. Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer,” *EMNLP*, 2018.
- [27] S. Chen et al., “Extending Context Window of Large Language Models via Position Interpolation,” *arXiv:2306.15595*, 2023.
- [28] B. Peng et al., “YaRN: Efficient Context Window Extension of Large Language Models,” *arXiv:2309.00071*, 2023.
- [29] bloc97, “NTK-Aware Scaled RoPE,” *GitHub issue #133*, 2023.
- [30] W. Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *JMLR*, 2022.
- [31] D. Lepikhin et al., “GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding,” *ICLR*, 2021.
- [32] DeepSeek-AI, “DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model,” *arXiv:2405.04434*, 2024.
- [33] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv:2501.12948*, 2025.
- [34] N. Razin et al., “Likelihood Displacement in Direct Preference Optimization,” *arXiv:2405.14853*, 2024.
- [35] G. Weiss et al., “Thinking Like a Transformer: A Theoretical Analysis of the Expressive Power of Transformers,” *ICLR*, 2021.
- [36] W. Merrill & A. Sabharwal, “The Parallelism Tradeoff: Limitations of Log-Precision Transformers,” *ICLR*, 2023.
- [37] O. Press et al., “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” *ICLR*, 2022.
- [38] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” *ICLR*, 2017.

- [39] Z. Zhou et al., “LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens,” *arXiv:2402.13753*, 2024.
- [40] T. Cai et al., “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads,” *ICML*, 2024.